INTERNSHIP TRAINING REPORT

ON

CLOUD COMPUTING USING AWS

ORGANISED BY

DREAMZ VISION FINISHING

TRAINER'S SIGNATURE

SUBMITTED BY

Priyanshu Durlav

NAME : Priyanshu Durlav

REG. NO. : D242535934

ROLL NUMBER : DKALCSTS5

SEM : Sem-5

SESSION : 2024-27

INSTITUTE NAME : KALNA POLYTECHNIC

FIELD OF TRAINING : CLOUD COMPUTING USING AWS

PERIOD : FROM 28/10/26 to 06/11/26

INTRODUCTION

Cloud computing is a model for delivering computing resources such as virtual servers, object storage, networking, security controls, and application services over the Internet on an on-demand, pay-as-you-go basis. Amazon Web Services (AWS) provides a comprehensive, highly secure, and globally distributed suite of cloud services that can be combined to architect, deploy, and maintain modern web applications with enterprise durability and zero on-premise hardware overhead.

This internship training project applies practical AWS cloud concepts to a full-stack, production-ready web application named **L'Aura Bistro — Digital Restaurant Menu Website**. The application functions as an interactive, luxury digital dining catalog allowing patrons to browse categorized gourmet food selections, conduct live dish searches, review dietary certifications (Vegetarian, Vegan, Gluten-Free), and stream high-definition culinary media clips.

The application architecture eliminates traditional database overhead by utilizing a structured, lightweight JSON data repository. The server logic is hosted on an **Amazon Elastic Compute Cloud (EC2)** virtual instance running Ubuntu Linux, managed continuously through **PM2** and reverse-proxied via **Nginx** on public HTTP Port 80.

All high-resolution culinary photography and promotional kitchen video assets are stored securely in a private **Amazon Simple Storage Service (S3)** bucket with Block Public Access fully enabled. Media files are dynamically streamed to visitors using temporary cryptographic **Presigned URLs** generated by the backend Express server.

The project demonstrates an end-to-end cloud workflow involving single-page React architecture, REST API communication, automated Linux process daemonization, **AWS IAM Instance Role** authorization (completely eliminating hardcoded credentials), and continuous GitHub-based cloud deployment.

AIM

To become familiar with core cloud computing concepts, AWS services, EC2 Linux virtual server hosting, Amazon S3 private object storage, IAM role-based permissions, REST API communication, automated media presigning, Nginx reverse proxy configuration, and the fundamental operations required to deploy and manage a secure, cloud-based web application.



Figure 1: Live L'Aura Bistro Digital Restaurant Menu application hosted on the AWS EC2 public endpoint (http://3.26.210.235/).

FAMILIARIZATION WITH CLOUD COMPUTING AND AWS

The training begins with the fundamental cloud concepts required to understand the project architecture:

1. **Cloud Computing:** On-demand access to virtualized computing resources, storage, and networking over the Internet.
2. **Amazon EC2:** Resizable virtual compute capacity used in this project to host the Node.js Express server and compiled React application.
3. **Amazon S3:** Secure, high-durability object storage used to store private culinary photos and promotional restaurant videos.
4. **AWS IAM:** Identity and Access Management used to securely govern permissions between the EC2 compute instance and the S3 bucket.
5. **IAM Role / Instance Profile:** Allows the EC2 application to obtain temporary, automatically rotated AWS credentials without storing permanent access keys on the server.
6. **Security Group:** Acts as a virtual firewall controlling inbound network traffic to the EC2 instance (allowing Port 80 HTTP and restricting Port 22 SSH).
7. **PM2 & Nginx:** PM2 maintains continuous backend uptime with auto-restart upon server reboot, while Nginx handles public Port 80 reverse-proxy routing.

The documented production deployment uses Ubuntu Server 24.04 LTS on EC2, Amazon S3 in the Sydney region (ap-southeast-2), Node.js 20, Express.js, React.js, Tailwind CSS, AWS SDK for JavaScript v3, PM2, and Nginx.

PROJECT OVERVIEW AND OBJECTIVES

Project Name: L'Aura Bistro — Digital Restaurant Menu Website

L'Aura Bistro is a modern, high-performance, single-page restaurant catalog application. It addresses the operational and financial challenges of traditional printed dining menus by providing an interactive digital showcase with live dish filtering, full-text ingredient searching, dietary classifications, kitchen promotional video streaming, and an administrative S3 cloud manager.

Primary Objectives

- Develop a responsive, mobile-first single-page web application using React.js, Vite, and Tailwind CSS.
- Eliminate database management complexities (MongoDB/MySQL) by utilizing a structured, reliable JSON data source (`data/menu.json`).
- Store high-resolution dish photography and promotional video clips in private Amazon S3 object storage with Block Public Access enabled.
- Implement secure, zero-credential authentication using an AWS EC2 IAM Instance Role, strictly avoiding hardcoded AWS Access Key IDs or Secret Access Keys.
- Generate temporary S3 Presigned GET URLs on the Express backend with a 1-hour expiration time to securely stream private media to authenticated clients.
- Provide a secure, PIN-protected Admin Manager allowing authorized staff to import starter menu media and upload new dish photos directly into Amazon S3.
- Host the full-stack system on an AWS EC2 Ubuntu instance using PM2 for process uptime and Nginx as a reverse proxy on Port 80.
- Establish an automated, repeatable GitHub-based continuous deployment workflow (`git push` locally → `git pull` & `pm2 restart` on EC2).

PROBLEM STATEMENT AND TECHNOLOGY STACK

Traditional restaurant operations rely heavily on physical paper menus that are prone to wear, expensive to reprint for seasonal dishes or price modifications, and incapable of showcasing multimedia. Conversely, complex digital menu systems frequently introduce database overhead (MongoDB/MySQL) and insecure credential handling.

This project resolves these challenges by coupling a lightweight JSON data structure with native AWS cloud infrastructure:

Layer / Tool	Purpose
HTML5 / React.js / Vite	Single-page application structure, state reactivity, and fast client bundling.
Tailwind CSS / Lucide	Modern luxury culinary styling, mobile-first responsive layout, and UI icons.
Node.js + Express.js	Backend REST API endpoints, static React asset hosting, and S3 Presigned URL signing.
Multer (Memory Storage)	In-memory buffering of uploaded dish photos for direct cloud streaming to S3.
AWS SDK v3 for JavaScript	Direct programmatic interaction with Amazon S3 using IAM Instance Role authentication.
Amazon EC2 (Ubuntu 24.04 LTS)	Virtual cloud compute hosting the Express server and static assets.
Amazon S3 (ap-southeast-2)	Private, durable cloud object storage for dish photos and promotional video files.
AWS IAM Instance Role	Role-based least-privilege permission management without permanent access keys.
Git / GitHub / npm / PM2 / Nginx	Version control, automated build, background process management, and Port 80 proxy.

MAIN APPLICATION FEATURES

- **Interactive Dish Filtering:** Instant tabbed filtering across Starters, Chef's Specials, Main Course, Desserts, and Beverages.
- **Real-time Food Search:** Live filtering by dish title, ingredients (e.g., truffle, ribeye, sea bass), or description text.
- **Dietary Badges:** Visual tags for Vegetarian, Vegan, Non-Vegetarian, and Gluten-Free with spiciness flame ratings.
- **Promotional Video Showcase:** HTML5 video player streaming kitchen craft videos directly from private S3 storage.
- **PIN-Protected Admin Manager:** Secure modal protected by a 4-digit PIN (`1234`) to import starter media and add new dishes.

AMAZON S3 STORAGE & DIRECT UPLOAD PROCEDURE

Amazon S3 serves as the primary object repository. The application maintains all image and video assets inside a private bucket (`expense-splitter-app-2026`) with Block Public Access fully enabled.

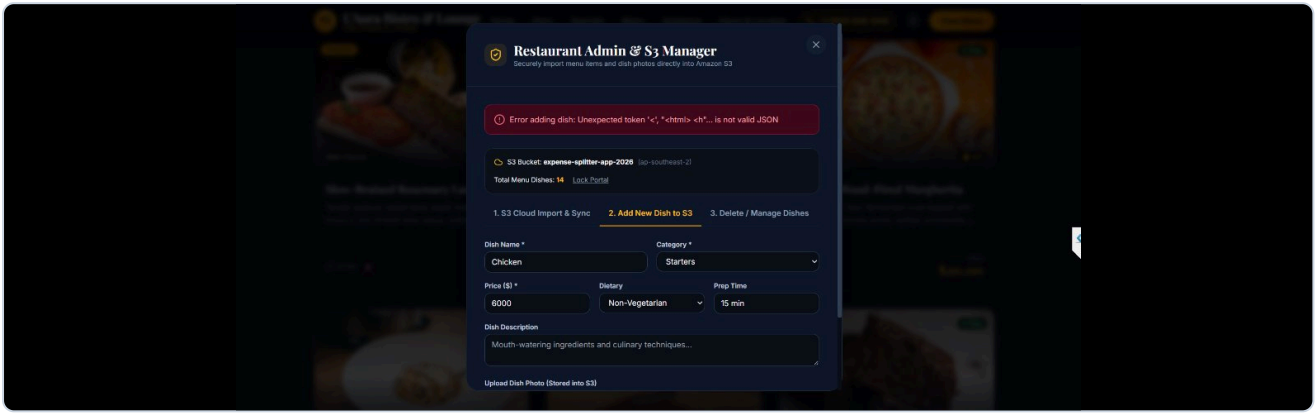


Figure 2: Restaurant Admin & S3 Manager interface on live application allowing direct dish creation and secure cloud photo upload.

DISH PHOTO UPLOAD & S3 INGESTION PROCEDURE

ADMIN OPENS ADMIN PORTAL & ENTERS SECURITY PIN (1234)

▼

INPUTS DISH DETAILS & SELECTS DISH PHOTO

▼

FRONTEND SENDS MULTIPART FORMDATA REQUEST

▼

NGINX REVERSE PROXY (CLIENT_MAX_BODY_SIZE 50M)

▼

EXPRESS RECEIVES FILE IN MEMORY BUFFER (MULTER)

▼

AWS SDK V3 PUTOBJECTCOMMAND STREAMS TO S3

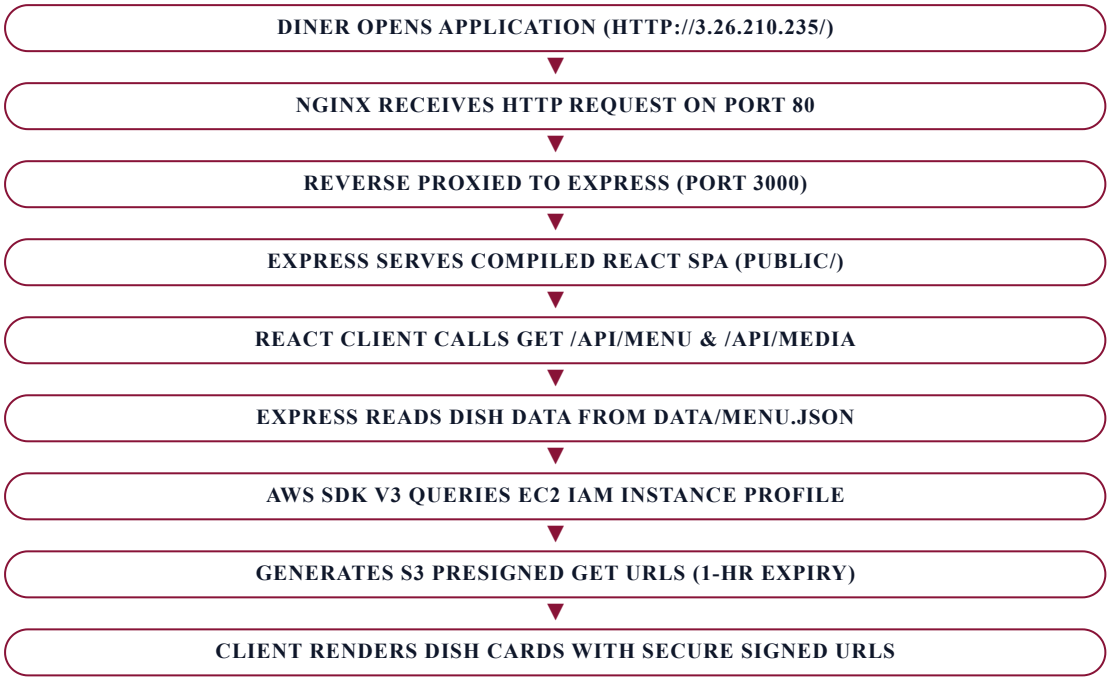
▼

S3 STORES OBJECT (IMAGES/DISHES/DISH-NAME.JPG)

▼

DISH RECORD APPENDED TO MENU.JSON & UI REFRESHED

COMPLETE APPLICATION WORKFLOW



SYSTEM ARCHITECTURE

The application follows a resilient Three-Tier Cloud Architecture:

1. Client Tier: Runs in the diner's web browser. Built with React.js and styled with Tailwind CSS, it executes search filtering, category selection, and video streaming without full page reloads.

2. Application Tier: Runs on an AWS EC2 Ubuntu instance. Express.js handles REST API endpoints, static React hosting, Multer file upload streaming, and S3 Presigned URL signing. Nginx manages Port 80 ingress and Gzip compression, while PM2 ensures process resilience.

3. Storage Tier: Amazon S3 stores private food photographs and promotional video files. Content remains non-public and is delivered exclusively through time-limited cryptographic presigned URLs.

AWS CLOUD ARCHITECTURE

USER BROWSER
React.js • Tailwind CSS
Live Search & Filters

AWS EC2
Ubuntu • Node.js
Express • PM2 • Nginx

AWS S3
Private Objects
images/dishes/*.jpg

AWS IAM ROLE (Instance Profile)
Zero Stored Access Keys • Dynamic Temporary Tokens

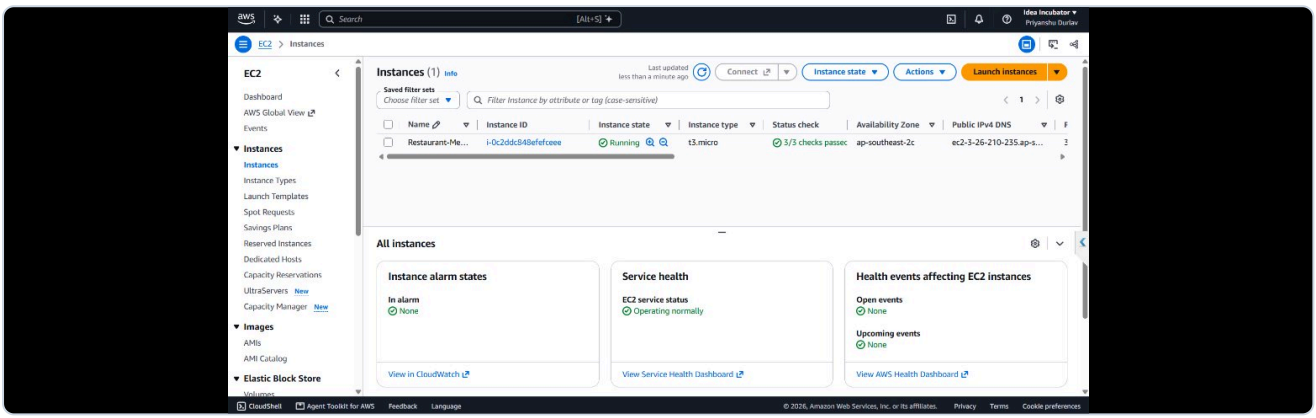


Figure 3: Amazon EC2 Console showing active Ubuntu instance Restaurant-Menu-Server (Instance ID: i-0c2ddc848efefceee, Public IP: 3.26.210.235, 3/3 checks passed).

PROJECT FOLDER STRUCTURE AND API

```
digital-restaurant-menu/
├── frontend/src/components/ (Navbar, FoodCard, CategoryFilter, SearchBar, AdminModal)
├── frontend/src/sections/ (HeroSection, IntroSection, MenuSection, PromoVideo, ...)
├── public/                  # Compiled production React SPA (served by Express)
├── server/config/s3.js      # AWS SDK v3 client authenticated via IAM Instance Role
├── server/controllers/      # menuController.js, mediaController.js, adminController.js
├── server/routes/           # menuRoutes.js, mediaRoutes.js, adminRoutes.js
├── data/menu.json           # 14 gourmet dish records & category metadata
├── server.js, ecosystem.config.js, nginx.conf, .env.example, README.md
```

BACKEND API DOCUMENTATION

Method & Endpoint	Auth	Function
`GET /api/health`	Public	Returns server uptime, health status, and S3 connectivity check.
`GET /api/menu`	Public	Retrieves menu items with dynamically resolved S3 Presigned URLs.
`GET /api/categories`	Public	Returns list of all active culinary categories.
`GET /api/media`	Public	Returns hero banner and promotional video Presigned URLs.
`POST /api/admin/verify-pin`	Public	Validates the 4-digit administrative security PIN (1234).
`POST /api/admin/sync-s3`	PIN Header	One-click automated seeding of all starter dish media to S3.
`POST /api/admin/add-dish`	PIN Header	Uploads dish photo to S3 and registers dish in menu.json.
`DELETE /api/admin/dish/:id`	PIN Header	Removes an item from menu.json.

AMAZON S3 STORAGE DESIGN

Amazon S3 provides cloud object storage with 99.999999999% (11 9's) data durability. The configured bucket is **expense-splitter-app-2026** in the Asia Pacific Sydney region (**ap-southeast-2**).

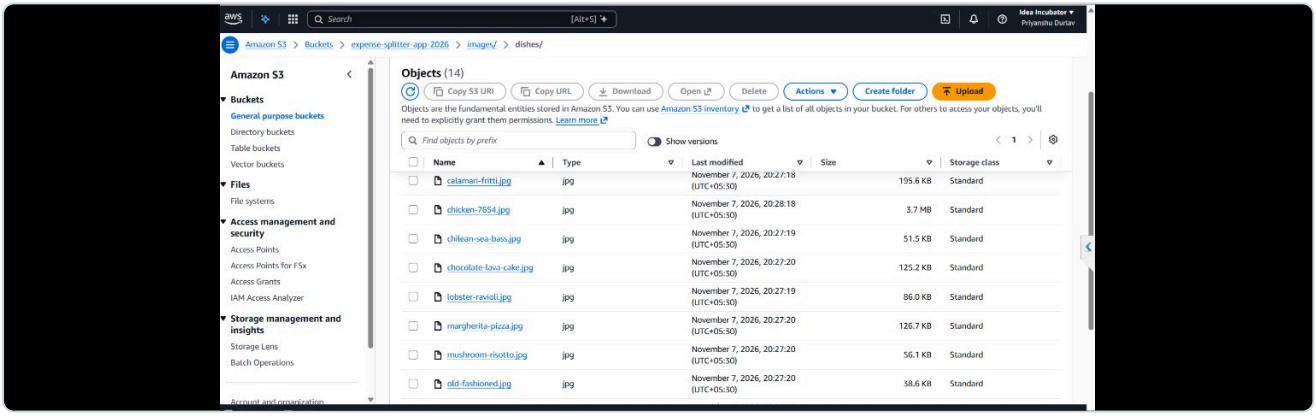


Figure 4: Amazon S3 Management Console displaying 14 active dish image objects stored in expense-splitter-app-2026/images/dishes/.

AWS IAM SECURITY CONFIGURATION

IAM controls which AWS resources the application can access. The EC2 instance assumes an IAM Instance Role (**EC2-Restaurant-S3-Role**), eliminating the security risk of storing hardcoded credentials.

Security Element	Project Configuration
IAM Role Name	EC2-Restaurant-S3-Role
Trusted Entity	AWS Service: EC2 (ec2.amazonaws.com)
Permissions Policy	Custom Inline Policy (RestaurantS3UploadPolicy)
Allowed Actions	s3:GetObject, s3:PutObject, s3:DeleteObject, s3:ListBucket
Target Resource	arn:aws:s3:::expense-splitter-app-2026/*
Credential Method	EC2 Instance Profile (Automatic temporary rotating tokens)
Permanent Access Keys	Not Required / Strictly Avoided
S3 Public Access	Block all public access enabled

AWS DEPLOYMENT PROCESS

1. **Step 1:** Create private S3 bucket `expense-splitter-app-2026` in `ap-southeast-2` with Block Public Access enabled.
2. **Step 2:** Create IAM Role `EC2-Restaurant-S3-Role` with S3 GetObject, PutObject, and ListBucket permissions.
3. **Step 3:** Launch an Ubuntu Server 24.04 LTS EC2 instance (`t3.micro`) and attach the IAM Role.
4. **Step 4:** Configure Security Group: Allow HTTP Port 80 from `0.0.0.0/0` and SSH Port 22 from personal IP.
5. **Step 5:** Connect to EC2 via SSH: `ssh -i restaurant-key.pem ubuntu@3.26.210.235`.
6. **Step 6:** Install Node.js 20 LTS, Git, PM2, and Nginx.
7. **Step 7:** Clone the GitHub repository: `git clone https://github.com/Prishu07/digital-restaurant-menu.git`.

AWS DEPLOYMENT PROCESS (CONTINUED)

Step 8: Install root backend dependencies and frontend dependencies:

```
cd digital-restaurant-menu
npm install
npm run install:frontend
```

Step 9: Compile the production React Single Page Application into the root `public/` folder:

```
npm run build
```

Step 10: Create the production environment configuration file (`.env`):

```
nano .env
PORT=3000
AWS_REGION=ap-southeast-2
S3_BUCKET_NAME=expense-splitter-app-2026
ADMIN_PIN=1234
```

Step 11: Start Express using PM2 process manager and configure automatic system restart:

```
pm2 start ecosystem.config.js
pm2 startup
pm2 save
```

Step 12: Configure Nginx as a reverse proxy with 50MB upload capacity to forward Port 80 to Port 3000:

```
sudo cp nginx.conf /etc/nginx/sites-available/restaurant-menu
sudo ln -s /etc/nginx/sites-available/restaurant-menu /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

Step 13: Open web browser and access the live application endpoint: `http://3.26.210.235/`.

DEPLOYMENT ARCHITECTURE AND PROJECT LINKS



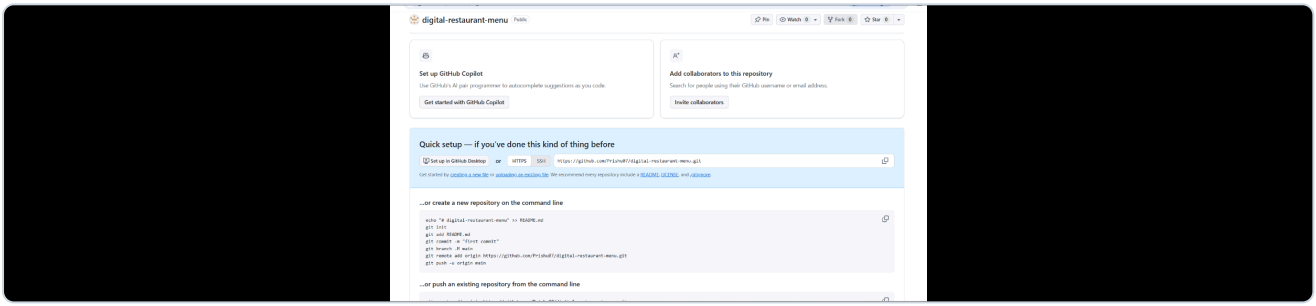


Figure 5: GitHub cloud repository setup hosting version-controlled project source code.

Live Application URL
http://3.26.210.235/

GitHub Repository URL
https://github.com/Prishu07/digital-restaurant-menu.git

TESTING AND VERIFICATION

Test	Action	Expected Result
Application Availability	Open EC2 Public IP URL (http://3.26.210.235/).	L'Aura Bistro restaurant UI loads with complete styling.
Health API	Execute `GET /api/health`.	Returns HTTP 200 with uptime and S3 configuration status.
Menu Retrieval	Execute `GET /api/menu`.	Returns 14 dishes with resolved temporary S3 Presigned URLs.
Category Filter	Click "Chef's Specials" category tab.	UI immediately filters to display only special culinary items.
Live Search	Type "Bruschetta" in search bar.	Instantly isolates Truffle Bruschetta dish without reload.
Promotional Video	Click Play in Promotional Video section.	HTML5 player streams kitchen video via S3 presigned URL.
Admin PIN Check	Enter PIN `1234` in Admin Modal.	Access granted to S3 media manager and dish editor.
S3 Media Import	Click "Start S3 Media Import" button.	Backend uploads starter dish photos directly to S3 bucket.
Direct Dish Upload	Upload custom photo and submit dish form.	File streams to S3 and registers dish in menu.json.
S3 Verification	Inspect S3 console at images/dishes/.	Uploaded dish files appear with correct byte sizes.
Security Verification	Inspect client network requests.	No AWS credentials exposed; S3 remains completely private.
PM2 Persistence	Execute `pm2 restart restaurant-menu`.	Application restarts in background with zero downtime.

Live production testing verified that all culinary assets stored in private bucket expense-splitter-app-2026 stream seamlessly in client browsers via presigned GET URLs without exposing AWS secrets or credentials.

SECURITY, COST, LIMITATIONS AND FUTURE IMPROVEMENTS

Security Considerations

- **S3 Block Public Access:** Fully enabled; direct unauthenticated public access to bucket objects is blocked.
- **IAM Least Privilege:** EC2 uses an IAM Instance Profile; no AWS access keys are stored in code or on disk.
- **Reverse Proxy Shield:** Internal Port 3000 is closed to the internet; Nginx inspects all traffic on Port 80.
- **Upload Guard:** Nginx enforces `client_max_body_size 50M;` to handle photos while blocking buffer floods.

Cost Considerations

- Built on AWS Free Tier eligible resources: `t3.micro` EC2 instance and standard S3 storage.
- In-memory backend caching of presigned URLs (45-min TTL) prevents redundant signing calculations.

Current Scope and Limitations

- **Included:** Single-page UI, live search, category filter, S3 presigned media streaming, PIN-protected admin manager, EC2 Ubuntu hosting with Nginx and PM2.
- **Not Included:** Multi-tenant customer logins, online table ordering/payment checkout, and automated SSL domain binding.

Possible Future Improvements

- Automated SSL/TLS certificate installation via Let's Encrypt Certbot (`https://`).
- Amazon CloudFront CDN integration for edge caching of high-resolution food photography.
- Online customer table reservation system and digital payment gateway integration.

FINAL PROJECT SUMMARY

L'Aura Bistro successfully demonstrates how modern cloud-native architecture can replace traditional database complexity with a lightweight, secure, and production-ready digital restaurant catalog. By combining React, Node.js, Amazon S3, and Amazon EC2 with IAM instance profile security, the project delivers enterprise-grade cloud performance with zero permanent credential exposure.

LEARNING OUTCOMES

- **Cloud Infrastructure Design:** Practical experience provisioning virtual compute (EC2) and object storage (S3) on AWS.
- **IAM Principle of Least Privilege:** Mastered IAM instance profiles to eliminate hardcoded AWS access keys.
- **Full-Stack Cloud Integration:** Successfully compiled a React.js single-page application into an Express backend.
- **Object Storage & Presigned URLs:** Implemented AWS SDK v3 cryptographic presigned URLs for secure private media delivery.
- **Production Server Administration:** Configured Nginx as a reverse proxy on Port 80 and established PM2 daemon process management on Ubuntu Linux.
- **DevOps Continuous Deployment:** Established a seamless Git/GitHub continuous update workflow for cloud applications.

FINAL ARCHITECTURE

